

北華航天工業學院

《人工智能应用开发实训》

课程设计报告

报告题目：_____

作者所在系部：_____

作者所在专业：_____

作者所在班级：_____

作者姓名：_____

指导教师姓名：_____

完成时间：_____ 2026 年 9 月

计算机学院

内容摘要

本文设计并实现了一个面向课程资料、论文与个人笔记的知识库问答系统。后端基于 FastAPI 与 LangChain，前端基于 React，以 Chroma 向量数据库持久化文档向量。针对扫描件与插图，引入 OCR 与视觉大模型生成图片描述；分块提供 recursive、token、character 三种策略；检索环节把稠密向量召回与 BM25 稀疏召回并行执行，经倒数排名融合后由 Cross-Encoder 精排，最终生成带文件、页码与章节引用的答案，并在资料不足时明确拒绝作答。系统以 JWT 令牌与 owner 元数据实现多用户数据隔离，配有 146 个后端单元测试；测试表明各模块运行正常、答案可溯源，达到设计要求。

关键词：检索增强生成；混合检索；倒数排名融合；交叉编码器重排；知识库问答

目 录

内容摘要	2
第 1 章 需求分析	5
1.1 项目背景和意义	5
1.2 项目主要完成内容	5
第 2 章 概要设计	7
2.1 系统功能框架	7
2.2 功能模块说明	7
第 3 章 详细设计	9
3.1 文档解析与入库模块	9
3.1.1 数据设计	9
3.1.2 算法描述	9
3.1.3 流程图	10
3.2 混合检索模块	11
3.2.1 数据设计	11
3.2.2 算法描述	12
3.3 问答生成模块	13
3.3.1 数据设计	13
3.3.2 算法描述	13
3.3.3 流程图	14
第 4 章 系统功能实现	15
4.1 抽象数据类型定义	15
4.1.1 文档片段	15
4.1.2 检索结果	15
4.1.3 会话消息	15
4.2 文档管理	16
4.2.1 函数说明	16
4.2.2 时间复杂度分析	16
4.3 检索问答	17
4.3.1 函数说明	17
4.3.2 时间复杂度分析	17
4.4 调用关系说明	18
第 5 章 系统测试	20
5.1 测试环境与测试用例设计	20
5.2 文档解析与入库功能测试	20

5.3 检索问答功能测试.....	21
5.4 异常处理与多用户隔离测试.....	21
5.5 测试结论.....	22
总 结.....	23
参考文献.....	24
指导教师评定意见.....	25

第1章 需求分析

1.1 项目背景和意义

大语言模型在自然语言理解与生成上表现突出，但在专业领域问答中容易产生“幻觉”——生成看似合理却缺乏依据的内容。与此同时，学习者日常面对的资料形态多样：课堂讲义多为 PDF，个人笔记多为 Markdown，习题与成绩表为 Excel，且分散在不同目录中，检索效率低。检索增强生成（Retrieval-Augmented Generation, RAG）把外部知识库与生成模型结合，先检索再生成，使答案有据可查，是缓解幻觉的有效途径。

本项目的意义有四点：其一，把分散的多格式学习资料统一入库，建立可检索的个人知识库；其二，通过稠密与稀疏的混合检索并配合重排序，提升召回的准确率与覆盖面；其三，约束模型只依据检索到的资料作答并给出引用来源，使结论可溯源、可核验；其四，通过多用户隔离，使系统能够用于班级或小组的共享场景。

1.2 项目主要内容

本实训完成了一个前后端分离的知识库问答系统，主要工作包括以下九个方面。

1. 多格式文档解析：支持 PDF、Markdown、TXT、DOCX、PPTX、XLSX 六类文件，统一转换为带元数据的文档对象。

2. 扫描件与插图的兜底处理：对文本量不足的 PDF 页面调用 OCR 识别，对页面内插图调用视觉大模型生成图片描述。

3. 文本分块：实现 recursive、token、character 三种分块策略，并支持块大小与重叠长度的参数化配置。

4. 向量化与持久化：基于嵌入模型生成文本向量并写入 Chroma 向量数据库，同时以 owner 元数据实现用户级隔离。

5. 混合检索：稠密向量召回与 BM25 稀疏召回并行执行，采用倒数排名融合（RRF）合并结果，再用 Cross-Encoder 精排。

6. 答案生成与引用：构造带编号的上下文提示词，调用大模型生成答案，并返回文件、页码、章节与片段等引用信息。

7. 多用户支持：用户注册登录、bcrypt 口令散列、JWT 令牌鉴权与四级数据隔离。

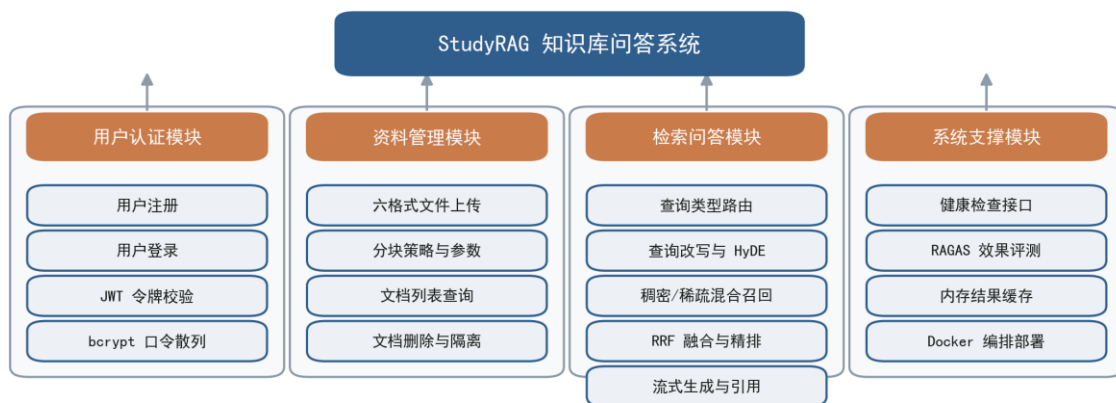
8. 交互界面：React 前端实现资料上传、分块策略选择、文档管理与流式问答。

9. 系统测试：编写 146 个后端单元测试，覆盖文档解析、分块、向量检索、接口与鉴权等环节。

第2章 概要设计

2.1 系统功能框架

系统采用前后端分离的浏览器/服务器结构。前端为 React 单页应用，开发态运行于 5173 端口，通过 HTTP 与 SSE 同后端通信；后端为 FastAPI 应用，运行于 8000 端口，负责鉴权、文档管理与问答编排；数据层由 Chroma 向量库与若干 JSON 登记文件构成。系统功能划分为用户认证、资料管理、检索问答与系统支撑四个模块，各模块及其子功能如图 2-1 所示。



2.2 功能模块说明

1. 用户认证模块

负责用户注册与登录。注册时校验用户名长度与字符集、口令长度，使用 bcrypt 对用户口令加盐散列后存入 users.json；登录成功后签发 HS256 算法的 JWT 令牌，有效期 7 天。其余业务接口通过 HTTPBearer 依赖解析令牌取得当前用户名，作为后续数据隔离的依据。

2. 资料管理模块

负责文档的接入与维护。上传接口对文件做扩展名与大小校验，按文件 sha256 与用户名的组合查重，随后调用解析器与分块器，把文本块写入向量库并重建 BM25 索引，最后登记元数据。列表接口按创建时间倒序返回当前用户的文档；删除接口按“向量→索引→哈希→登记→磁盘文件”的顺序清理，并校验文档归属，非本人文档返回 403。

3. 检索问答模块

系统的核心模块。提问后先执行查询类型路由与查询改写，得到不超过 3 条查询；对每条查询分别执行稠密召回与稀疏召回，用 RRF 融合后取前若干条候选送入 Cross-Encoder 精排；最终把 Top-K 片段格式化为带编号的上下文，交由大模型生成答案。该模块提供同步与 SSE 流式两种接口。

4. 系统支撑模块

包括健康检查接口、基于 RAGAS 的效果评测接口、内存缓存以及 Docker Compose 编排配置，用于运行状态观测、问答效果评估与部署。

第3章 详细设计

3.1 文档解析与入库模块

3.1.1 数据设计

1. 逻辑结构设计

入库过程处理的基本数据对象是文档片段 `Document`，其逻辑结构可表示为：

`Document = ( content , metadata )`

其中 `content` 为片段正文，`metadata` 为描述该片段的元数据集合，各字段的含义与取值如表 3-1 所示。

表 3-1 文档片段元数据字段说明

字段名	类型	含义
<code>document_id</code>	字符串	文档唯一标识，取 <code>uuid4</code> 的十六进制串
<code>filename</code>	字符串	原始文件名
<code>file_type</code>	字符串	文件类型，取值为 <code>pdf</code> 、 <code>markdown</code> 、 <code>txt</code> 、 <code>docx</code> 、 <code>pptx</code> 、 <code>excel</code> 之一
<code>page</code>	整数	页码，PDF 与 PPTX 从 1 开始计数
<code>chapter</code>	字符串	所属章节标题，由标题样式或 Markdown 标题解析得到
<code>chunk_index</code>	整数	片段在文档内的序号，从 0 开始
<code>chunk_strategy</code>	字符串	分块策略，取值为 <code>recursive</code> 、 <code>token</code> 、 <code>character</code> 之一
<code>owner</code>	字符串	所属用户名，用于多用户数据隔离

2. 存储结构设计

向量与元数据写入 Chroma 的 `collection`「`studyarag_docs`」，距离度量取余弦相似度（`hnsw:space = cosine`），索引结构为 HNSW 近邻图，数据以嵌入式模式持久化到本地目录。由于 Chroma 0.5.x 只接受字符串、整数、浮点与布尔类型的元数据，取值为 `None` 的 `page` 与 `chapter` 字段在写入前需要剔除。

文档登记信息、用户信息与文件哈希分别存放于 `documents.json`、`users.json` 与 `file_hashes.json` 三个 JSON 文件，写入时以线程锁保护，避免并发写坏文件。上传的原始文件保存在 `data/uploads/` 目录下，文件名替换为 `uuid4` 的十六进制串，并以原扩展名结尾，从而避免文件名冲突与路径穿越。

3.1.2 算法描述

解析算法按扩展名分派到不同的解析器，各有侧重。

（1）PDF 解析。逐页解析为一条文档对象，页码由零基换算为从 1 开

始。若某页去除空白后的字符数少于阈值（默认 50），判定为扫描件，则将该页以 300 DPI 渲染为位图并调用 OCR 识别，用识别结果替换页面文本；随后抽取该页的内嵌图像，调用视觉大模型生成图片描述，按「[图片 i 描述] …」的形式追加到页面文本之后，使图表内容也能被检索到。

（2）Markdown 解析。用正则匹配一至六级标题，按标题把正文切分为若干章节，标题文字作为 chapter 元数据；首个标题之前的内容归入「前言」。

（3）DOCX、PPTX、XLSX 解析。分别用 python-docx、python-pptx 与 openpyxl 完成：DOCX 依据标题样式判定章节，样式缺失时退化为「前言」与「第 N 节」；PPTX 按幻灯片逐页提取文本框、表格和备注；XLSX 按工作表逐表提取，每行以竖线拼接为一条记录，工作表名作为章节。

分块算法提供三种策略，可按文件类型与资料特点选择。

（1）recursive 递归分块（默认）。按分隔符优先级由粗到细递归切分，分隔符随文件类型变化：PDF 使用空行、换行与句号等，Markdown 优先在二至四级标题和代码块边界切分，Excel 以行与竖线为界。该策略尽量保持语义完整，是通用性最好的一种。

（2）token 分块。以 tiktoken 的 token 数为切分单位，重叠长度按块大小的十分之一折算，适合需要严格控制上下文体量的场景。

（3）character 分块。以固定字符数并配合双换行分隔符切分，实现简单、行为可预期。

切分完成后为每个片段写入 chunk_index 与 chunk_strategy 元数据。向量化时调用嵌入模型对片段批量编码，按每批 10 条写入向量库，随后把全部片段加入 BM25 索引，最后登记文档元数据并记录文件哈希。

3.1.3 流程图

文档解析入库的流程图如图 3-1 所示。

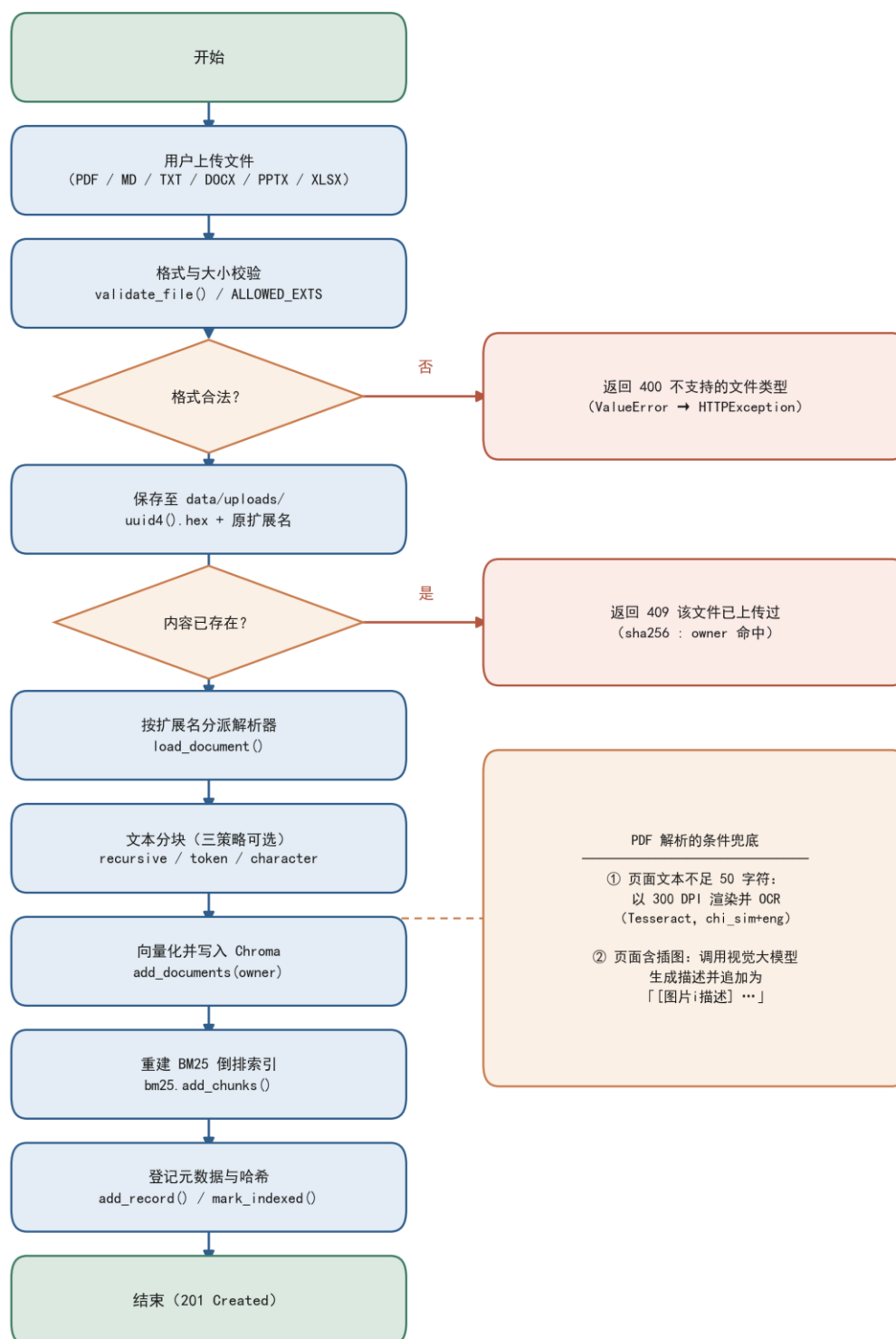


图 3-1 文档解析入库流程图

3.2 混合检索模块

3.2.1 数据设计

检索结果统一表示为 `SearchResult` 数据类，字段包括：`content`（片段正文）、`score`（相关性得分）、`filename`（来源文件名）、`page`（页码）、`chapter`（所

属章节)、document_id (文档标识)、chunk_index (片段序号)、source (来源, 取 dense、sparse、fusion、rerank 之一) 与 metadata (原始元数据)。对外输出时附加 excerpt 字段, 取正文前 200 个字符作为摘要, 并将 score 四舍五入保留四位小数, 便于前端展示与比较。

3.2.2 算法描述

1. 查询路由

用大模型把问题归入事实查询、概念解释、对比分析、总结概括四类之一, 每类对应一组检索参数: 事实查询取 $\text{top_k} = 6$; 概念解释取 4; 对比分析取 8; 总结概括取 3 且关闭稀疏召回。分类失败时默认按概念解释处理, 保证可用性。

2. 查询改写

与路由并行执行两项改写。一是同义改写: 展开缩写、补充领域上下文并保持原意, 最多产出 2 条陈述句查询。二是 HyDE: 让模型先「假设」一段 2 至 4 句的答案, 再以该假设答案的向量去检索, 从而缩小问题与文档之间的表述差异。两项改写与原问题合并后截取前 3 条, 作为多路召回的查询集合。

3. 稠密召回

调用嵌入模型把查询编码为向量, 在 Chroma 上按余弦相似度做近邻搜索, 检索条数取 $\text{recall_k} = \text{top_k} \times 3$, 并以 owner 作为过滤条件。由于 Chroma 返回的是余弦距离, 需换算为相似度: $\text{相似度} = 1 - \text{距离}$, 并截断到 $[0, 1]$ 区间。

4. 稀疏召回

用 jieba 对语料分词并建立 BM25Okapi 索引。检索时对查询分词后计算各片段的 BM25 得分, 取前 $2 \times \text{top_k}$ 条候选, 按 owner 过滤后做 min-max 归一化, 把得分映射到 $[0, 1]$ 区间, 以便与稠密召回的得分处于同一量纲。

5. 倒数排名融合

两条召回路径的结果用倒数排名融合 (RRF) 合并。对候选片段 d, 融合得分为:

$$\text{RRF}(d) = \sum_i \frac{1}{(k + \text{rank}_i(d))}$$

其中 $\text{rank}_i(d)$ 是 d 在第 i 路结果中的排名 (从 1 开始), k 取常数 60, 用于削弱头部排名的绝对优势、让更多路径的结果有机会进入候选集。去重键取正文前 80 个字符与文件名的拼接, 得分相同时保留原始得分较高者。

6. 交叉编码器精排

取融合结果的前 6 条候选，构造「查询，片段正文」对送入 Cross-Encoder 模型，模型对每一对做一次完整的前向计算并输出相关性分数，按新分数降序排列后返回前 top_k 条。与双塔向量模型相比，交叉编码器让查询与文档在注意力层充分交互，精度更高，但计算量随候选数线性增长，因此只用于小规模候选集。

检索流程如图 3-2 所示。

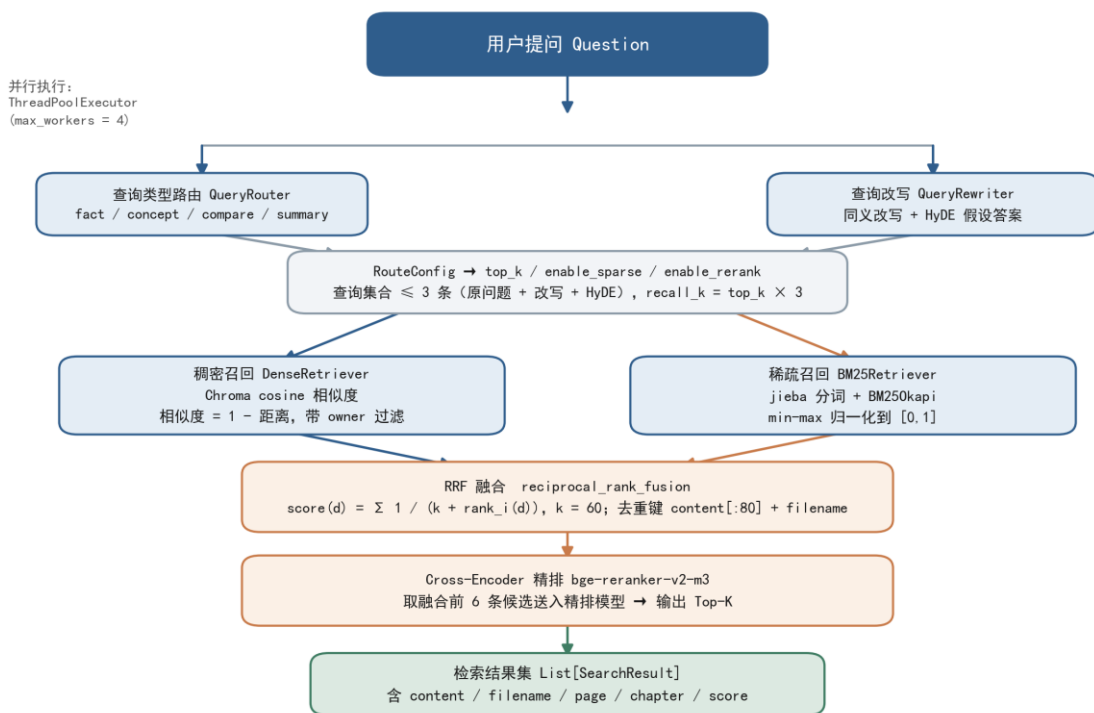


图 3-2 多路召回与 RRF 融合流程图

3.3 问答生成模块

3.3.1 数据设计

提示词由系统模板与用户模板两部分组成。系统模板规定四条约束：只能依据参考资料作答；引用处标注编号；资料不足时回复固定话术；使用简体中文输出。用户模板包含 `context` 与 `question` 两个占位符。上下文按「[i]（来源：文件名，第 N 页，「章节」）」的格式为每条片段编号，便于模型在答案中标注引用。最终问答响应包含 `answer`、`sources` 与 `question` 三个字段，其中 `sources` 为引用列表，每项含文件名、页码、章节、片段摘录与得分。

3.3.2 算法描述

生成算法的步骤为：拼接带编号的上下文与用户问题，填入提示词模板；

调用大模型，温度取 0.1 以降低输出的随机性；把模型输出解析为答案文本；再把检索结果转换为引用列表一并返回。流式接口在路由、改写、召回、精排、生成各阶段向队列写入进度事件，并在生成阶段逐块读取模型的流式输出，包装为 SSE 事件实时下发，使前端能够展示检索进度与逐字生成的答案。

3.3.3 流程图

问答生成的流程如图 3-3 所示。

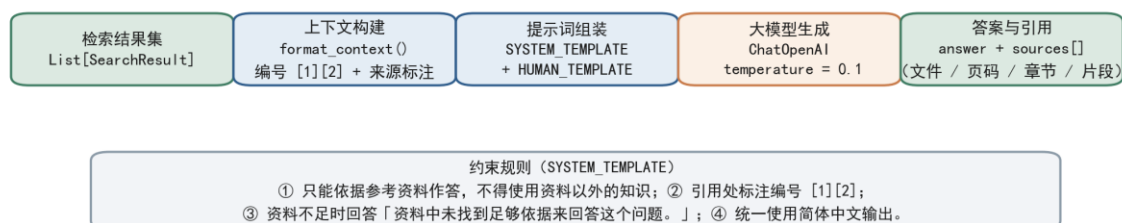


图 3-3 问答生成流程图

第4章 系统功能实现

4.1 抽象数据类型定义

4.1.1 文档片段

文档片段是本系统最基本的数据单位，其抽象数据类型定义如下。

ADT Document

{

数据对象: $D = \{ d_i \mid d_i = (content_i, metadata_i), i = 1, 2, \dots, n, n \geq 0 \}$

数据关系: $R = \{ \langle d_{i-1}, d_i \rangle \mid d_{i-1}, d_i \in D \text{ 且 } chunk_index_i = chunk_index_{i-1} + 1 \}$

基本操作:

load_document(path, document_id)

初始条件: path 指向一个受支持且可读的文件。

操作结果: 解析文件, 返回按页或按章节组织的 Document 列表。

split_documents(docs, strategy, size, overlap)

初始条件: docs 非空, strategy $\in \{ recursive, token, character \}$ 。

操作结果: 按策略与参数切分, 返回带 chunk_index 的片段列表。

add_documents(chunks, owner)

初始条件: chunks 非空, 嵌入服务可用。

操作结果: 把片段向量写入向量库, 并在元数据中注入 owner。

delete_by_document_id(document_id)

初始条件: document_id 对应的向量已存在。

操作结果: 删除该文档的全部片段向量, 返回删除数量。

} ADT Document

4.1.2 检索结果

检索结果封装了召回内容与它的来源信息, 其抽象数据类型定义如下。

ADT SearchResult

{

数据对象: $S = (content, score, filename, page, chapter,$

$document_id, chunk_index, source, metadata)$

数据关系: 同一次检索返回的结果集合按 score 降序排列。

基本操作:

to_dict()

初始条件: 无。

操作结果: 转为字典, 附加 excerpt = content[:200], score 保留四位小数。

rerank(query, candidates, top_k)

初始条件: candidates 非空且规模较小 (实现中限制为前 6 条)。

操作结果: 用交叉编码器重新打分并降序排列, 返回前 top_k 条。

} ADT SearchResult

4.1.3 会话消息

前端以消息列表维护一次会话的上下文, 其抽象数据类型定义如下。

ADT Message

{

数据对象: $M = (role, content, sources)$

数据关系: 列表中的消息按发生时间先后排列, 用户消息与助手消息交替出现。

基本操作:

ask_question(question, owner)

初始条件: owner 的知识库中至少已索引一份文档。

操作结果: 返回 ChatResponse, 含 answer、sources 与 question。

ask_question_stream(question, owner, on_progress, on_done)

初始条件: 同上。

操作结果: 按 SSE 协议依次回调进度事件与最终结果事件。

} ADT Message

4.2 文档管理

4.2.1 函数说明

文档管理相关的核心函数说明如表 4-1 所示。

表 4-1 文档管理模块主要函数说明

函数名	所在文件	功能说明	主要参数	返回值
validate_file	utils/file_utils.py	校验扩展名与文件大小上限	filename, size	布尔值, 不合法则抛异常
save_upload_file	utils/file_utils.py	分块流式写盘, 重命名为 uuid 串	upload_file, dir	落盘后的文件路径
is_duplicate	utils/hash_store.py	以 sha256 与 owner 组合判定重复	file_path, owner	布尔值
load_document	services/loader.py	按扩展名分派解析器, 含 OCR 与 VLM 兜底	file_path, document_id	Document 列表
split_documents	services/splitter.py	按策略切分并写入片段序号	docs, strategy, size, overlap	片段列表
add_documents	services/vectorstore.py	分批写入向量库并注入 owner	chunks, owner	无
add_chunks	retrievers/bm25_retriever.py	重建 BM25 索引并登记语料	chunks	无
add_record / list_records	utils/registry.py	登记与查询文档元数据	record, owner	无 / 记录列表
delete_by_document_id	services/vectorstore.py	删除某文档的全部片段向量	document_id	删除数量

4.2.2 时间复杂度分析

设文档字符总数为 n , 切分得到的片段数为 m , 单次嵌入请求的批大小为 $B1 = 10$, 文档原有片段数为 N 。

文件校验与哈希计算需要对文件做一次完整遍历, 时间复杂度为 $O(n)$;

解析阶段按页或按工作表线性遍历，同为 $O(n)$ ，但 OCR 与视觉大模型调用属于外部服务，其耗时取决于页数，记为 $O(p)$ (p 为触发兜底的页数)。

分块阶段，递归分块对每个字符做常数比较，复杂度为 $O(n)$ ；向量化需要对 m 个片段各做一次嵌入计算，按每批 10 条写入向量库，因此写入次数为 $\lceil m/B \rceil$ ，单个片段的向量写入经 HNSW 近似为 $O(\log N)$ 。

BM25 索引重建需要对语料重新分词，设语料总长度为 L ，分词复杂度为 $O(L)$ ，故该步骤为 $O(N + m)$ (含新加入的片段)。删除操作需要先按 `document_id` 检索出全部待删片段，再逐条删除，复杂度为 $O(N)$ 。

4.3 检索问答

4.3.1 函数说明

检索问答相关的核心函数说明如表 4-2 所示。

表 4-2 检索问答模块主要函数说明

函数名	所在文件	功能说明	主要参数	返回值
<code>get_config</code>	<code>services/router.py</code>	问题分类并返回检索参数	<code>query</code>	<code>RouteConfig</code> 字典
<code>rewrite</code>	<code>services/query_rewriter.py</code>	并行执行同义改写与 HyDE	<code>question</code> , <code>enable_hyde</code>	查询字符串列表
<code>retrieve</code>	<code>retrievers/dense_retriever.py</code>	向量近邻检索并换算相似度	<code>query</code> , <code>top_k</code> , <code>owner</code>	<code>SearchResult</code> 列表
<code>retrieve</code>	<code>retrievers/bm25_retriever.py</code>	BM25 打分并 min-max 归一化	<code>query</code> , <code>top_k</code> , <code>owner</code>	<code>SearchResult</code> 列表
<code>reciprocal_rank_fusion</code>	<code>services/fusion.py</code>	多路结果按倒数排名融合	<code>result_sets</code> , <code>k</code> , <code>final_top_k</code>	融合后的结果列表
<code>rerank</code>	<code>services/reranker.py</code>	交叉编码器重排序	<code>query</code> , <code>candidates</code> , <code>top_k</code>	重排后的结果列表
<code>format_context</code>	<code>services/prompt.py</code>	把结果格式化为带编号的上下文	<code>retrieved</code>	上下文字符串
<code>_full_search_sync</code>	<code>services/chain.py</code>	串起路由、改写、召回、融合与精排	<code>query</code> , <code>top_k</code> , <code>event_q</code> , <code>owner</code>	最终结果列表
<code>ask / ask_stream</code>	<code>services/chain.py</code>	同步与流式的完整问答	<code>question</code> , <code>owner</code>	<code>ChatResponse</code> / 生成器

4.3.2 时间复杂度分析

设查询条数为 Q (实现中 $Q \leq 3$)，单次召回条数为 `recall_k`，向量库片段总数为 N ，语料总 token 数为 L ，融合后候选数为 M ，精排候选数为 C (实现中 $C \leq 6$)。

查询路由需要一次大模型调用，查询改写需要两次 (同义改写与 HyDE)，

路由与改写并行执行，因此该阶段耗时近似为一次最慢调用的耗时。

稠密召回依赖 HNSW 近邻图，单次查询的复杂度约为 $O(\log N)$ ，对 Q 条查询共需 Q 次，即 $O(Q \cdot \log N)$ 。稀疏召回中 BM25 打分为 $O(|Q_terms| \cdot avg_dl \cdot N)$ ，其中 $|Q_terms|$ 为查询词项数、 avg_dl 为平均文档长度；由于实现中对候选做了截断，实际开销小于全量打分。

融合阶段对两路结果去重后排序，复杂度为 $O(M \log M)$ ，其中 M 与 $recall_k$ 同阶。精排阶段受限于候选数上限 C ，需要 C 次交叉编码器前向计算，每次前向的复杂度与输入长度成线性关系，记为 $O(C \cdot (|q| + |d|))$ ；由于 C 被限制在 6 以内，精排是检索链路中延迟占比最高的环节，但总体开销可控。

4.4 调用关系说明

系统的分层调用关系为：路由层接收请求并校验身份，把业务委托给服务层；服务层负责流程编排，向下调用检索器、融合、重排与提示词模块；最底层为 Chroma 向量库、交叉编码器与外部大模型服务。主要函数之间的调用关系如图 4-1 所示。

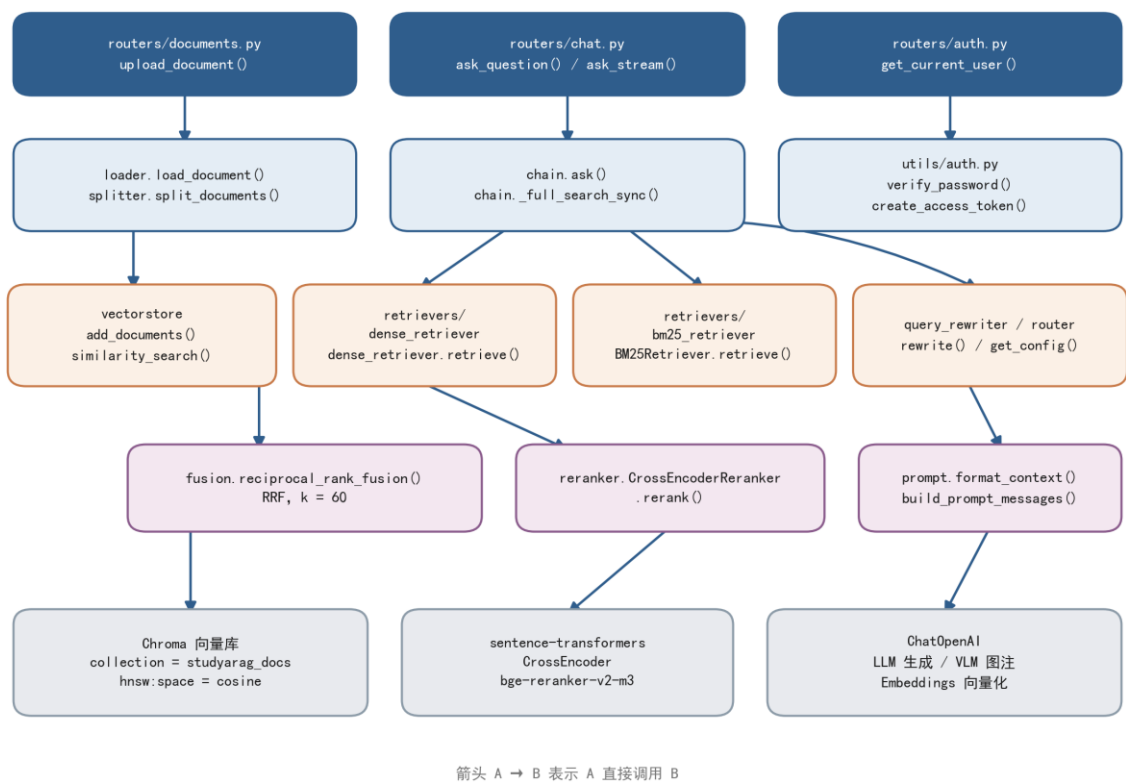


图 4-1 函数间的调用关系

第 5 章 系统测试

5.1 测试环境与测试用例设计

测试在 Windows 11 环境下进行，后端使用 Python 3.11 与 pytest 8.3.4，通过 httpx 的 ASGI 传输直接调用 FastAPI 应用对象，无需启动真实服务器。接口测试统一以依赖覆盖的方式把鉴权依赖替换为固定的测试用户，从而把测试重点集中在业务逻辑本身。

测试用例按被测对象划分为 11 个文件，共 146 个用例，覆盖文档解析、分块、向量库、提示词、检索、链路编排、接口与鉴权等模块，用例分布如表 5-1 所示。

表 5-1 测试用例分布

测试文件	用例数	主要测试内容
test_loader.py	40	六类文件的解析结果、元数据字段、页码与章节抽取、异常输入
test_splitter.py	31	三种分块策略、分隔符选择、元数据继承、边界情况
test_vectorstore.py	13	写入、相似度检索、按文档删除、集合统计
test_documents_api.py	12	上传、列表、删除接口与健康检查
test_prompt.py	12	上下文格式化与提示词组装
test_auth.py	10	注册、登录、口令校验与令牌鉴权
test_chat_api.py	8	问答接口的答案、引用与错误分支
test_embeddings.py	7	嵌入服务配置与单例行为
test_retriever.py	7	检索结果字段、摘要长度与空结果处理
test_chain.py	3	问答链路的结果结构
test_health.py	3	健康检查与前置条件校验

5.2 文档解析与入库功能测试

选取一页含公式与插图的课程讲义 PDF、一份带多级标题的 Markdown 笔记、一份含三张工作表的 Excel 成绩表与一份演示文稿进行上传测试。结果显示：六类文件均能正确解析，PDF 的页码从 1 开始连续编号，Markdown 的章节标题被正确抽取为 chapter 元数据，Excel 的工作表名成为章节名。对一页纯图片的扫描件，系统自动触发 OCR，识别出的文字被写回页面文本并成功入库，该页的插图也生成了图片描述。

分块策略的对比测试表明：同一份讲义在 recursive 策略下得到 16 个片段，在 token 策略下片段数明显增多且长度更均匀，在 character 策略下片段长度严格受 chunk_size 约束。三种策略产出的片段数互不相同，说明策略参

数确实生效。

文档上传成功后的界面如图 5-1 所示。

（此处插入：文档上传成功后的界面截图）

图 5-1 文档上传与入库结果

5.3 检索问答功能测试

以上传的《自然语言处理》讲义为知识库，依次提出概念解释型、事实查询型与对比分析型问题各若干条，观察答案内容与引用来源是否正确。

概念解释型问题（如「什么是 Transformer？」）返回了分点作答的结果，并在句末标注了引用编号；展开引用后可以看到来源文件、页码与所属章节，例如「第 3 页 • 2.1 自注意力机制」，与讲义原文相符。对比分析型问题触发了 $\text{top_k}=8$ 的路由配置，召回片段数与答案覆盖度均高于概念解释型。

针对资料外的提问（如询问与知识库无关的物理概念），系统未编造答案，而是回复「资料中未找到足够依据来回答这个问题。」，并提示当前知识库仅包含相应课程资料，说明提示词中的约束规则生效。

问答结果与引用展开的界面如图 5-2 所示。

（此处插入：问答结果与引用来源截图）

图 5-2 问答结果与引用来源展示

5.4 异常处理与多用户隔离测试

异常处理测试覆盖了输入不合法与运行期异常两类情形，结果如表 5-2

所示。所有异常分支均返回了明确的提示信息，且未在服务端留下残留文件。

表 5-2 异常处理测试结果

测试项	输入	期望结果	实际结果
不支持的格式	上传 .exe 文件	返回 400 并提示不支持的文件类型	符合预期
重复上传	两次上传同一文件	第二次返回 409	符合预期
缺少文件字段	不携带 file 字段	返回 422 校验错误	符合预期
空知识库提问	未上传文档即提问	返回 400 并提示先上传资料	符合预期
问题为空	question 传空串	返回 422 校验错误	符合预期
非法令牌	携带伪造的 JWT	返回 401 并提示凭证无效	符合预期
令牌过期	携带过期的 JWT	返回 401 并提示重新登录	符合预期
越权删除	删除他人文档	返回 403 并提示不属于自己	符合预期

多用户隔离测试使用两个账号分别上传同名但内容不同的文件进行验证。结果显示：两个账号都能上传成功（哈希查重以「文件摘要：用户名」为键），各自的文档列表中只能看到本人上传的文档；用账号 A 的令牌检索时，向量库以 owner 字段过滤，无法召回账号 B 的片段；尝试删除账号 B 的文档时返回 403。四级隔离均符合预期。

多用户隔离测试的界面如图 5-3 所示。



图 5-3 多用户数据隔离验证

5.5 测试结论

上述测试表明：系统的文档解析、分块、向量化、检索、重排、生成与多用户隔离功能均按设计正常工作；146 个后端单元测试全部通过；异常分支返回了明确的错误码与提示信息；答案均附带可核验的引用来源，且在资料不足时能够拒绝作答。系统的功能与可靠性达到设计要求。需要说明的是，OCR 与视觉大模型、交叉编码器的效果受外部服务与本地模型权重影响，在有网络且模型权重完整的条件下表现最佳。

总 结

本次实训完整实现了一个检索增强生成的知识库问答系统。首先是理解了 RAG 的完整链路：以往只知道「向量检索加提示词」，动手实现后才发现效果取决于每一环——解析质量决定入库内容是否正确，分块粒度决定检索单元的语义是否完整，召回策略决定是否遗漏相关片段，提示词约束决定模型是否会脱离资料自由发挥。

其次是认清了混合检索与重排序的价值：单纯依靠向量检索时，专有名词与缩写常召回不准，引入 BM25 稀疏召回后，关键字精确匹配补齐了短板；两路结果用倒数排名融合合并，可避免得分量纲不一致带来的偏差；再用交叉编码器对少量候选精排，在精度与延迟之间取得平衡。

实训中的主要问题及解决办法：一是向量库只接受字符串、整数与布尔类型的元数据，页码为空时传入 `None` 会报错，改为写入前净化元数据；二是批量写入过大会超时，改为每批 10 条；三是扫描件与插图解析不出文字，加入 OCR 与视觉大模型兜底；四是交叉编码器在离线环境无法加载，改为下载权重后按本地路径加载。

设计上仍可改进：检索参数与重排序候选数目前是固定配置，可改为可配置项并依据命中反馈调整；会话历史尚未持久化，可增加会话与消息存储；大文件上传仍是同步阻塞，可改为任务队列加进度查询；评测所需的问答数据集尚未固化，应补充并纳入持续集成；配置中的密钥应改用环境变量注入，避免随代码提交。

参考文献

- [1] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]//Advances in Neural Information Processing Systems (NeurIPS), 2020: 9459-9474.
- [2] Robertson S, Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond[J]. Foundations and Trends in Information Retrieval, 2009, 3(4): 333-389.
- [3] Cormack G V, Clarke C L A, Buettcher S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods[C]//Proceedings of the 32nd International ACM SIGIR Conference, 2009: 758-759.
- [4] Nogueira R, Cho K. Passage Re-ranking with BERT[J]. arXiv preprint arXiv:1901.04085, 2019.
- [5] Gao L, Ma X, Lin J, et al. Precise Zero-Shot Dense Retrieval without Relevance Labels[C]//Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), 2023: 1762-1777.
- [6] Karpukhin V, Oguz B, Min S, et al. Dense Passage Retrieval for Open-Domain Question Answering[C]//Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020: 6769-6781.
- [7] Malkov Y A, Yashunin D A. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2020, 42(4): 824-836.
- [8] Es S, James J, Espinosa-Anke L, et al. RAGAS: Automated Evaluation of Retrieval Augmented Generation[C]//Proceedings of the 18th Conference of the European Chapter of the ACL (EACL), 2024: 150-158.
- [9] 严蔚敏, 吴伟民. 数据结构 (C 语言版) [M]. 北京: 清华大学出版社, 2007.
- [10] Chase H. LangChain: Building Applications with LLMs through Composability[EB/OL]. <https://github.com/langchain-ai/langchain>, 2024.
- [11] Chroma. Chroma: The AI-native Open-source Embedding Database[EB/OL]. <https://github.com/chroma-core/chroma>, 2024.
- [12] FastAPI. FastAPI Documentation[EB/OL]. <https://fastapi.tiangolo.com>, 2024.

指导教师评定意见

指导教师评语：

成绩评定：

（签名）

年 月 日